

CLAUDE.md

Behavioral guidelines to reduce common LLM coding mistakes. Merge with project-specific instructions as needed.

Tradeoff: These guidelines bias toward caution over speed. For trivial tasks, use judgment.

1. Think Before Coding

Don't assume. Don't hide confusion. Surface tradeoffs.

Before implementing:

- State your assumptions explicitly. If uncertain, ask.
- If multiple interpretations exist, present them - don't pick silently.
- If a simpler approach exists, say so. Push back when warranted.
- If something is unclear, stop. Name what's confusing. Ask.

2. Simplicity First

Minimum code that solves the problem. Nothing speculative.

- No features beyond what was asked.
- No abstractions for single-use code.
- No "flexibility" or "configurability" that wasn't requested.
- No error handling for impossible scenarios.
- If you write 200 lines and it could be 50, rewrite it.

Ask yourself: "Would a senior engineer say this is overcomplicated?" If yes, simplify.

3. Surgical Changes

Touch only what you must. Clean up only your own mess.

When editing existing code:

- Don't "improve" adjacent code, comments, or formatting.
- Don't refactor things that aren't broken.
- Match existing style, even if you'd do it differently.
- If you notice unrelated dead code, mention it - don't delete it.

When your changes create orphans:

- Remove imports/variables/functions that YOUR changes made unused.
- Don't remove pre-existing dead code unless asked.

The test: Every changed line should trace directly to the user's request.

4. Goal-Driven Execution

Define success criteria. Loop until verified.

Transform tasks into verifiable goals:

- "Add validation" → "Write tests for invalid inputs, then make them pass"
- "Fix the bug" → "Write a test that reproduces it, then make it pass"
- "Refactor X" → "Ensure tests pass before and after"

For multi-step tasks, state a brief plan:

```
1. [Step] → verify: [check]
2. [Step] → verify: [check]
3. [Step] → verify: [check]
```

Strong success criteria let you loop independently. Weak criteria ("make it work") require constant clarification.

These guidelines are working if: fewer unnecessary changes in diffs, fewer rewrites due to overcomplication, and clarifying questions come before implementation rather than after mistakes.

Commands

```
# Run all three apps concurrently (web, server, desktop)
npm run dev

# Run individual apps
npm run dev:web      # Vite dev server → http://localhost:5173
npm run dev:server   # nodemon → http://localhost:4000
npm run dev:desktop  # Electron Forge

# Build / lint web
npm run build -w apps/web
npm run lint -w apps/web

# Package desktop app (produces .exe)
npm run make -w apps/desktop
```

There are no automated tests configured in this repo.

Architecture Overview

EvoDraw is a real-time collaborative whiteboard. The monorepo ([npm workspaces](#)) has three apps:

App	Tech	Purpose
apps/web	React 19 + Vite + Fabric.js	Browser whiteboard client

App	Tech	Purpose
apps/server	Express 5 + Socket.IO + MongoDB	Relay, persistence, auth
apps/desktop	Electron + Fabric.js	Screen-annotation overlay

Data flow

```

web client —Socket.IO→ server —Socket.IO broadcast→ other web clients
                                                    ↳ desktop overlay
web client —REST→ server → MongoDB (room state)
                               ↳ Firebase Storage (images)
web client —LiveKit SDK→ LiveKit SFU (voice/video/screen tracks)
desktop ————— launched via evodraw:// deep link from web

```

apps/server

`src/server.js` is the entry point — Express + Socket.IO share one HTTP server. Socket handlers are registered in `src/sockets/`:

- `room.handler.js` — join/leave room, user list
- `draw.handler.js` — canvas ops, cursor positions, snapshots
- `chat.handler.js` — chat messages and LiveKit token generation
- `screen.handler.js` — screen share session lifecycle
- `overlay.handler.js` — desktop overlay stroke relay

Authentication uses JWT (24h expiry, secret `TOKEN_SECRET`). Room passcodes are bcrypt-hashed in MongoDB. Rooms auto-delete after 24h via a MongoDB TTL index on `updatedAt`.

Required .env in apps/server/:

```

PORT=4000
MONGODB_URI=
TOKEN_SECRET=
ALLOWED_ORIGINS=http://localhost:5173
LIVEKIT_API_KEY=
LIVEKIT_API_SECRET=
LIVEKIT_URL=
FIREBASE_SERVICE_ACCOUNT_PATH=./firebase-service-account.json
FIREBASE_STORAGE_BUCKET=

```

apps/web

`src/pages/RoomPage/RoomPage.jsx` is the root component; all feature logic lives in hooks under `src/hooks/`. The component tree is thin — hooks own all state.

Key hooks:

Hook	Responsibility
<code>useRoom.js</code>	Socket.IO connection lifecycle, user list
<code>useCanvasSync.js</code>	Serialize/deserialize Fabric ops over Socket.IO
<code>useDrawingTools.js</code>	Fabric brush, shapes, arrows, eraser
<code>useHistory.js</code>	Undo/redo with sceneVersion
<code>useLiveKitRoom.js</code>	LiveKit SFU connection for voice/video
<code>useScreenShare.js</code>	Screen capture + DOM video + Fabric proxy rect
<code>useOverlayStrokes.js</code>	Receives desktop overlay strokes and renders them
<code>useRemoteCursors.js</code>	Other users' cursor positions
<code>useInfiniteCanvas.js</code>	Pan/zoom viewport
<code>useImagePasting.js</code>	Clipboard paste → Firebase upload → Fabric image

Vite env vars expected (prefix `VITE_`): `VITE_SERVER_URL`, `VITE_API_URL`.

Canvas Sync — LWW conflict resolution (`src/utils/canvasSerializer.js`)

Every Fabric object carries three metadata properties:

- `_evId` — stable UUID (timestamp + counter + random)
- `_evoVersion` — integer bumped on every local mutation
- `_evoNonce` — random tiebreaker for same-version conflicts

`shouldAcceptRemote(local, remote)`: accept if `remote._evoVersion > local._evoVersion`, or if equal and `remote._evoNonce < local._evoNonce` (deterministic, lower nonce wins).

`attachSerializer()` hooks Fabric events (`object:added/modified/removed`) and emits `canvas_op` Socket.IO events. `applyRemoteOp()` applies incoming ops using LWW reconciliation without re-emitting. The `state._applying` flag prevents echo loops.

Special object flags:

- `_evoScreenShare: true` — proxy rect for a video overlay; excluded from snapshots by default
- `_evoImage: true` — uploaded image; immune to eraser
- `_evoDrawing: true` — in-progress shape, skipped by serializer

Periodic snapshot push (every 10s when `canvas._evoIsDirty`) via `save_snapshot` event stores elements + `sceneVersion` in MongoDB.

Screen Sharing (two-layer rendering)

1. **Bottom**: native `<video>` DOM element positioned over the canvas absolutely
2. **Top**: near-invisible Fabric proxy `Rect` (`fill: 'rgba(0,0,0,0.005)'`) that makes the video region draggable/resizable

The proxy rect syncs its position to the DOM video element on every Fabric render tick (`syncOverlayPosition()`). LiveKit SFU delivers tracks to viewers; when `TrackSubscribed` fires the viewer calls `setupOverlay()` to mount its own `<video>` and locate or create the proxy rect.

apps/desktop

An Electron app that launches as an always-on-top transparent overlay. Launched from the web app via `evodraw://overlay?room=...&shareId=...&token=...` deep link.

- `src/main.js` — main process: single-instance lock, deep-link handler, tray icon, mode toggle (Working = click-through / Drawing = interactive), global hotkey (Ctrl+Shift+D)
- `src/renderer/overlay.js` — Fabric canvas fullscreen, draws strokes, emits `overlay:stroke:add` with normalized coordinates (0–1 range)
- `src/preload.js` — context isolation bridge (IPC between main and renderer)

Overlay strokes use normalized coordinates so they scale to any screen resolution.

Socket.IO Event Reference

Direction	Event	Purpose
client→server	<code>join_room</code>	Enter a room
server→clients	<code>room_users</code>	Updated user list
client→server	<code>canvas_op</code>	Single element add/modify/remove
server→clients	<code>canvas_op_received</code>	Broadcast canvas op to other clients
client→server	<code>save_snapshot</code>	Persist full canvas to MongoDB
client→server	<code>canvas_state_request</code>	Request snapshot from a peer
peer→peer	<code>canvas_state_response</code>	Peer sends snapshot
client→server	<code>livekit:get-token</code>	Request LiveKit JWT (callback)
client→server	<code>screen:start</code> / <code>screen:stop</code>	Screen share session
client→server	<code>overlay:stroke:add</code>	Desktop overlay stroke
server→clients	<code>overlay:stroke:added</code>	Relay overlay stroke to web clients